Where the Fix Lives: Code-Graph Structure and the Triage Gap for Local Coding Agents

Haresh G. (Kaggle: Haresh276) · Code and data: github.com/Haresh276/where-the-fix-lives

Abstract

A local agent such as Gemma 4 31B fixes few bugs, and under a fixed 12-hour budget its failures also take the most time. I ask which properties of a bug's fix location predict failure, and at what point an agent could know them. I build static code graphs of 12 Python repositories at the base commit of all 500 SWE-bench Verified tasks, on CPU, and join them with per-task outcomes for 38 models run in one scaffold family (mini-SWE-agent; refits within each release agree). Two properties predict failure beyond patch size and issue length: an issue that names nothing connected to the fix in the graph (−0.91 [−1.46, −0.40] log-odds) and a fix that touches more files (−0.34 [−0.65, −0.10] per SD). Multi-file fixes drop weak models from 30% to 7.4% resolved. None of this can be read from the issue: predicting a weak model's solves before search is at chance (AUC 0.45–0.54), and knowing where the true fix sits adds little over a cheap BM25 hit. What separates wins from losses is the *extent* of the edit, meaning how many files and functions it spans (AUC 0.63–0.72). On public run logs, capping the predicted-hard half of tasks at a model's median solve cost loses 11–21% of its solves when ranked by that extent, against 23–25% when capped at random. In my own run of Gemma 4 31B on all 129 public competition tasks on the competition's 4×L4 setup, it solves 36% of solvable one-function fixes and 15% of multi-function fixes, and 42% vs 11% split at a 10-line patch ($p < 0.001$). The usable rule is an edit-plan checkpoint, not a pre-screen.

1. Introduction

The Gemma 4 Developer Agent competition gives an agent 12 hours on four L4 GPUs to fix a hidden set of bugs. In my run of Gemma 4 31B on the competition's public tasks, failing tasks took twice as long as solved ones (median 364 s vs 180 s) and used 80% of total runtime. If an agent could tell early which bugs it will not fix, it could move that time to bugs it can.

Patch size is the obvious predictor, and it matters. But two three-line patches can differ completely. In one, the issue names the broken function. In the other, the broken line is three calls away from anything the issue mentions, in a file the issue never names. A repository's code graph makes that difference measurable, and the competition gives every agent graph tools.

I test three questions:

1. Does **where the fix sits in the code graph** predict failure beyond patch size and issue length, across models?
2. Can an agent read that signal **before** it searches, or only once it has found and planned the fix? I call the difference the *triage gap*.
3. Does the pattern hold for **Gemma 4 31B on the competition's own tasks**?

Prior work predicts task difficulty from LLM-judged task features [2], and shows on synthetic call-graph tasks that code understanding, not editing, limits coding agents [3]. Localization-first pipelines [7] and graph-guided localizers [8] treat finding the fix site as its own stage. I ask what the fix site, once found, says about whether the agent will succeed. I measure real repository structure instead of asking a model to judge it, and I compare against [2]'s released features.

2. Data and method

Tasks and outcomes. SWE-bench Verified [4] has 500 human-validated bug-fix tasks from 12 Python repositories; Django is 46% of them. The public leaderboard publishes per-task results for 40 runs of the bash-only *mini-SWE-agent* scaffold [5]. I drop two runs whose per-task files disagree with their reported score by more than 5 points (Claude 3.7 Sonnet, Gemini 3 Pro), leaving **38 models** (15 open-weight) resolving 9%–77%. The runs span 11 releases of the scaffold, so model and scaffold version are not fully separated. Each run gets its own fixed effect, and Section 3.1 refits within single releases. I call the five models below 45% *weak* (Qwen2.5-Coder 32B, Llama 4 Maverick, gpt-oss-120b, GPT-5 nano, Kimi K2) and the 13 at or above 70% *strong*.

Code graphs at every commit. For each task I parse every non-test Python file at its base commit, read from a bare git clone and cached by file hash, into definitions (functions, methods, classes). Definitions are linked by call edges and by containment edges (class to method). A call resolves by name to every definition with that name, which over-approximates, as most static Python call graphs do. Graphs have a median of 9,308 nodes (maximum 31,488).

Where the fix lives. I map each task's curated fix spans to graph nodes (97.5% resolve) and compute, among others:

- *hops*: shortest graph distance from any definition the issue names (identifier tokens, traceback frames; names with more than five definitions are ignored) to any fixed definition. 0 means the issue names the fixed function. *No anchor* (72 tasks) means no usable name in the issue reaches the fix within 8 hops; for 61 of these the issue names nothing in the graph at all;
- *files* and *functions* touched by the fix;
- whether the issue names the fix *file*;
- controls: patch lines and issue length.

Example. In `sphinx-doc__sphinx-9591`, the issue reports that a property's type annotation is not cross-referenced. The fix changes three lines in one method, `PyProperty.handle_signature`, three hops from the nearest definition the issue names. All 13 strong models fix it and none of the five weak models do. Patch size alone would call it easy.

Analyses. (a) A pooled logistic model of 19,000 task × model outcomes with a fixed effect per run and standardized features; 95% CIs from 1,000 task-level bootstrap resamples, plus leave-one-repository-out refits. (b) The same features fitted per model. (c) Ridge regression of item-response difficulty from [2], against [2]'s LLM-judged features. (d) Classifiers of each weak model's successes at increasing information levels, cross-validated with whole repositories held out (Section 3.3).

Competition tasks. The competition's public tasks are merged pull requests in fastapi, requests, rich and httpx, named `<repo>_<PR number>`. I ran Gemma 4 31B (QAT, the competition's default agent and tools, vLLM on 4×L4) on all 129 of them in one Kaggle session: 6 minutes, 40 tool calls and 70 turns per task, 10.8 GPU-hours in total. I rebuilt each task from public git history (fix commit and its parent) and derived its fix spans from the diff. On 150 SWE-bench tasks this diff mapping matches the curated files in every case, with 91% precision and 83% recall at function level. 27 fastapi tasks have tests that import packages missing from the sandbox, as the forum reports [6]; Gemma solved 0 of them, and I exclude them, leaving 102. As a cross-check I use another entrant's public per-task outcomes on 55 of the same tasks [1].

3. Results

3.1 Where the fix sits predicts failure

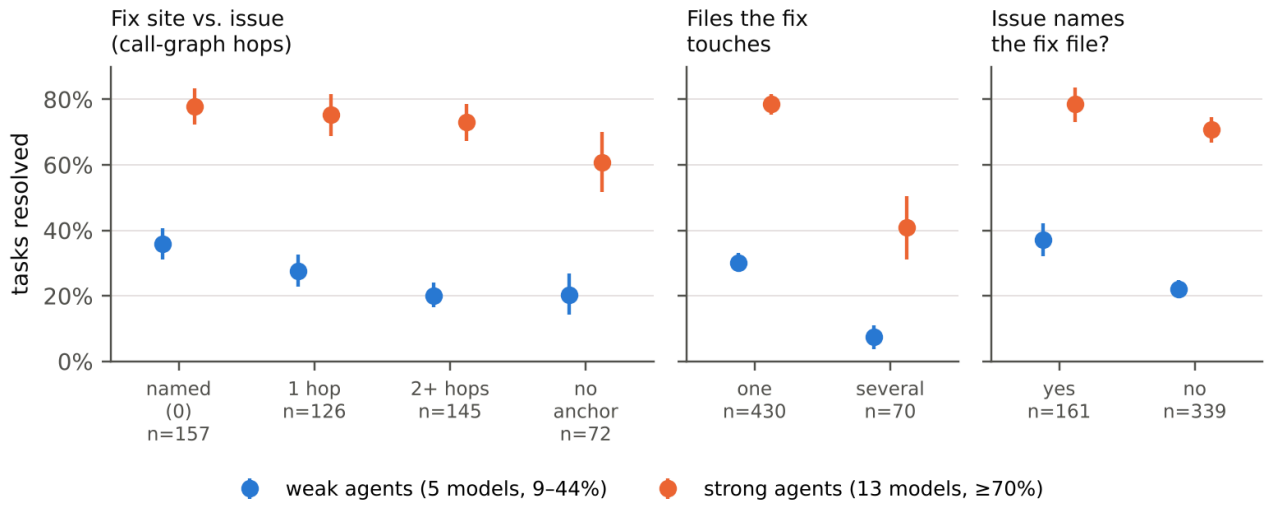


Figure 1. Resolve rate by where the fix lives, for weak and strong models (mean over models; 95% CIs from resampling tasks). SWE-bench Verified, 500 tasks.

Holding patch size, issue length and the other features fixed, the strongest structural signal is the absence of an anchor. When no name in the issue reaches the fix, the log-odds of success change by −0.91 [−1.46, −0.40] relative to an anchored task (odds ×0.40). Among anchored tasks, distance itself points the same way but is not distinguishable from zero (−0.15 [−0.36, 0.04] per SD). (Coding "no anchor" as distance 9 gives −0.28 [−0.46, −0.11] but mixes the two effects.) The result does not depend on edge choice: using call edges only, without containment, gives −0.27 [−0.43, −0.11] (rank correlation with the main distance 0.92).

Files touched carries its own penalty (−0.34 [−0.65, −0.10] per SD, independent of patch lines). An issue that names the fix file helps (0.15 [−0.01, 0.33]). How the fix is spread across functions, once files and patch size are controlled, adds nothing (−0.01 [−0.22, 0.17]).

These estimates are stable. Hops keep their sign with any single repository removed (12/12) and without Django (−0.28); files weakens without Django (−0.23). Within single scaffold releases, both effects keep their sign: v2.0.0 (11 models) gives hops −0.31 [−0.56, −0.08] and files −0.34 [−0.73, −0.06], and v1.7.0, which holds three of the five weak models, gives −0.21 [−0.37, −0.08] and −0.44 [−0.79, −0.15].

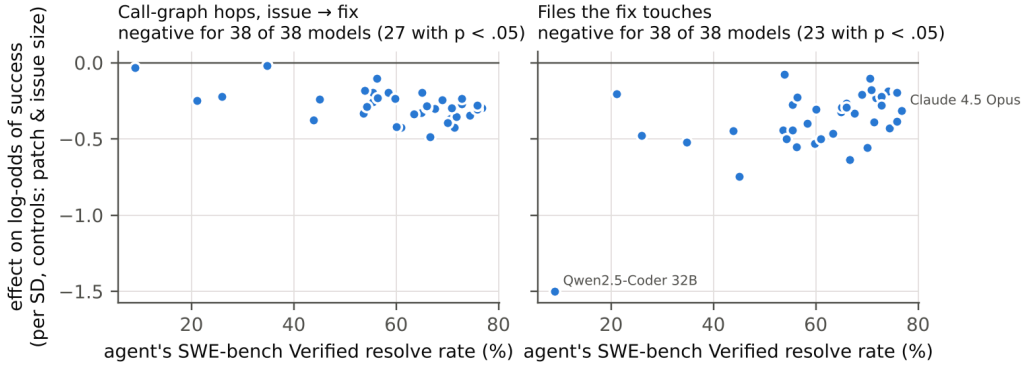


Figure 2. Per-model effect of hops (left) and files touched (right) on log-odds of success, controlling patch and issue size. Each dot is one of 38 models.

Fitted per model, hops is negative for 38 of 38 models and significant for 27 (Figure 2). The per-model fits share the same 500 tasks, so this is a consistency check, not 38 replications. Shuffling hops across tasks still gives all 38 the same sign 2% of the time.

In raw rates, weak models resolve 36% of tasks whose issue names the fixed function and 20% of those two or more hops away. For multi-file fixes they reach **7.4%** [3.7%, 11.1%], against 30% for single-file fixes.

Do weak models suffer more? Not on the log-odds scale. Multi-file fixes cost weak models 1.68 log-odds and strong models 1.67, the same penalty. The weak cohort's larger relative drop (75% vs 48%) is what a shared penalty produces at a lower base rate: structure makes tasks hard for every model, and weak models sit lower on that axis.

3.2 Static structure vs. an LLM judge

Table 1. Predicting item-response difficulty of the 500 tasks (Spearman ρ, 5-fold CV; 95% bootstrap CI).

Features	ρ	leave-repo-out ρ
Patch size only	0.40 [0.32, 0.47]	0.39
Code-graph features (ours)	0.42 [0.34, 0.49]	0.38
Graph + patch size	0.49 [0.42, 0.55]	0.46
LLM-judged features [2]	0.69 [0.64, 0.73]	0.64
LLM-judged + graph + patch	0.70 [0.65, 0.74]	0.64

This is a negative result. For *generic* difficulty, an LLM that reads the issue, patch and tests beats free static structure, and my features add almost nothing on top. The graph predicts something narrower: which known fixes weak models miss.

3.3 The triage gap

Could an agent skip its hopeless tasks early? For each weak model I predict which tasks it solves, holding out whole repositories, at five information levels:

1. the issue only (names, traceback, length, repository size);
2. plus a CPU BM25 localizer's top hit and the graph around it;
3. plus where the true fix sits (hops, whether its function or file is named);
4. plus its extent (files, functions);
5. plus the true patch size.

Levels 3–5 use the true fix and are oracles. Level 4 is the information an agent has once it has planned a correct edit.

Before search, prediction is at or below chance (AUC 0.45–0.54). The BM25 localizer, which ranks the true fix file first 44% of the time and in its top five 75%, lifts it to 0.52–0.62. Knowing exactly where the true fix sits barely helps beyond that (0.52–0.64). The jump comes with the fix's extent: 0.63–0.72, and 0.64–0.76 with true patch size. Patch size alone reaches 0.62–0.71. **The signal is in how big the edit is, not in where it is.**

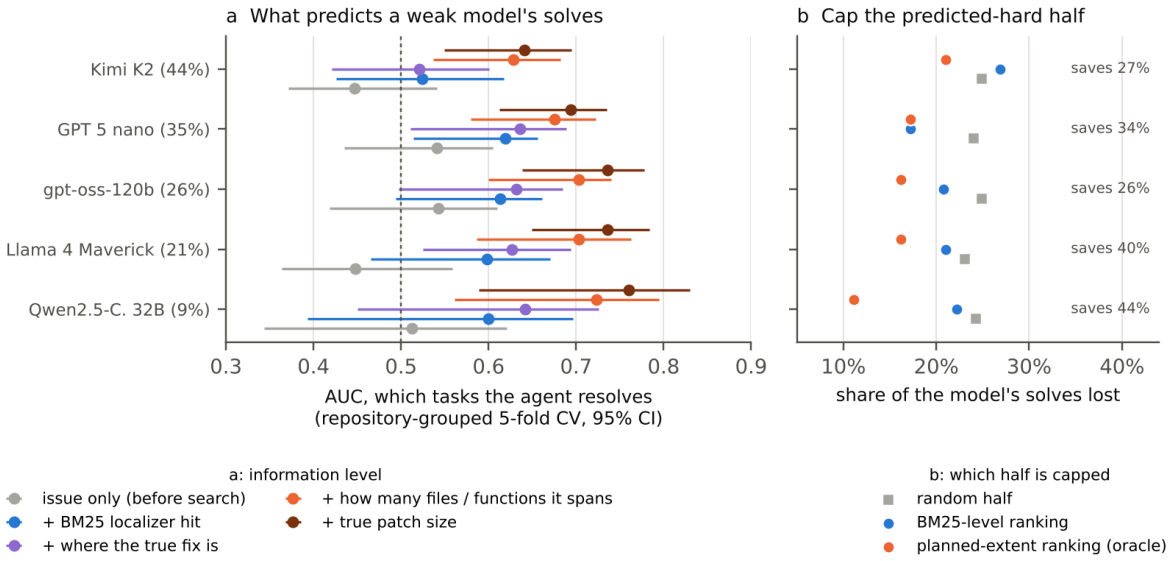


Figure 3. (a) AUC for predicting which tasks each weak model resolves, by information level (repository-grouped 5-fold CV; 95% CI from refitting on task resamples). (b) Share of solves lost when the predicted bottom half of tasks is capped at the model's median calls per solved task.

Simulating a checkpoint. The leaderboard logs record how many calls each run spent on each task. I declared one rule before looking: rank tasks by predicted solvability, cap the bottom half at the model's median calls per solved task, and count a solve as lost if it needed more calls than the cap. Ranked by planned extent (level 4), the cap saves 26–44% of all calls and loses 11–21% of solves. Capping a random half saves 22–38% and loses 23–25%. BM25-level information loses 17–27%, which beats random for four of five models (Figure 3b). It prices the loss; it cannot show what the saved calls would buy.

The budget at stake is large. Weak models spend 64–97% of their calls on tasks they fail; if every task cost the same, that share would be 56–91%.

Table 2. Share of a weak model's calls and solves in the bottom half of predicted solvability.

Model (resolve rate)	**Calls on failed tasks**	**Before search: calls / solves**	**Planned extent: calls / solves**
Qwen2.5-Coder 32B (9%)	97%	52% / 40%	56% / **22%**
Llama 4 Maverick (21%)	92%	45% / 62%	54% / **26%**
gpt-oss-120b (26%)	79%	51% / 48%	54% / **29%**
GPT-5 nano (35%)	85%	51% / 47%	47% / **35%**
Kimi K2 (44%)	64%	50% / 56%	56% / **40%**

3.4 Gemma 4 31B on the competition's tasks

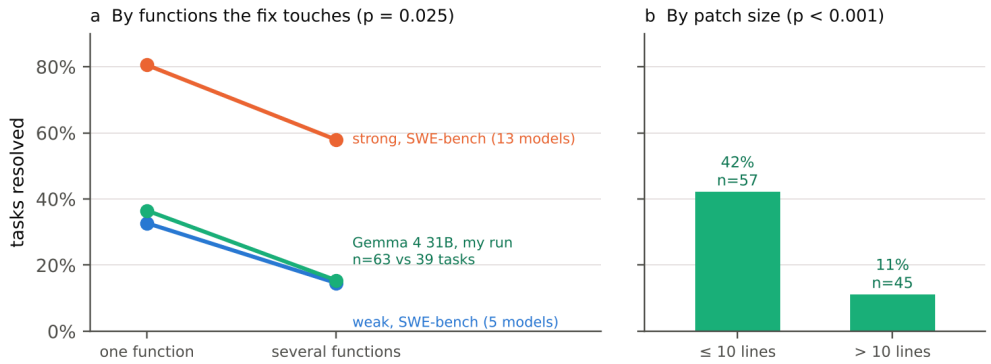


Figure 4. My Gemma 4 31B run on the 102 solvable public competition tasks. (a) By functions the fix touches, with the SWE-bench cohorts from Figure 1. (b) By patch size.

Gemma 4 31B resolves 29 of 102 solvable tasks (28%). Fixes inside one function succeed **36%** of the time (n = 63), and fixes spanning several functions **15%** (n = 39; Fisher p = 0.025). That is the same drop as the SWE-bench weak cohort (Figure 4a). The split holds after dropping tasks whose fix touches no Python definition (38% vs 15%, p = 0.022). Patch size separates the tasks more sharply: 42% for patches of 10 lines or fewer against 11% above (p < 0.001). Fitted together, patch size carries the effect (−0.64 per log-line, p = 0.024) and function count adds nothing (p = 0.979). Multi-file alone is weaker (32% vs 16%, p = 0.133). This matches Section 3.3: what predicts Gemma's failures is the extent of the fix.

The failures are expensive. 56 of 129 tasks hit the 6-minute limit and 10 overflowed the 32k context. Multi-function tasks time out 54% of the time against 30% for one-function tasks. Capping tasks with a patch over 10 lines at the median solve time (180 s) would save 22% of wall-clock time and lose 17% of solves. A random cap on the same number of tasks saves 19% and loses 21%, so on one run the edge is modest. The other entrant's CPU runs give the same picture on their 55 tasks (50% vs 17% on 41 solvable) [1].

4. What this means for a 12-hour budget

1. **Do not pre-screen from the issue.** Before search there is nothing to learn, and a skip rule at that point mostly skips solvable tasks.
2. **Checkpoint on the planned edit.** After locating the code and before editing, the agent should state how many files and functions it will change. A multi-file or multi-function plan is the low-yield half. For such a plan, make the smallest edit at the single most likely site, run one test, submit, and move on. A submitted partial patch costs nothing.
3. **Spend the time where it pays.** Under a binding 12-hour cap, less time on low-yield tasks lets `max_time_minutes` rise for all.

I did not run this checkpoint inside the agent; the simulations in Sections 3.3 and 3.4 are the evidence. This complements a harness study finding that call quality beats call volume for Gemma 4 [9]: it says which tasks to stop spending calls on.

5. Limitations

The SWE-bench outcomes come from one scaffold family across 11 releases. Within-release refits agree, but run settings and dates still differ, and SWE-bench tasks may be memorized. Levels 3–5 of Figure 3 use the true fix. An agent must estimate its own edit's extent, and the gap between that estimate and the truth is untested. Name-based call resolution over-approximates edges, and the competition's own graph tools may differ from mine. My Gemma

run is one sample per task on 102 tasks from only four repositories. Another entrant reports 34.5% resolved locally against about 6.9% on the hidden board [1], so local rates do not transfer directly to private repositories.

6. Release

Apache-2.0 at github.com/Haresh276/where-the-fix-lives:

- the commit-exact graph builder;
- the diff-to-span mapper;
- per-task features for all 500 SWE-bench Verified tasks and all 129 public competition tasks;
- my per-task Gemma 4 31B outcomes on the 129 tasks and the notebook that produced them;
- one script (`paper_numbers.py`) that regenerates every number in this paper from public data on CPU.

AI-use statement: an AI coding assistant (Claude) was used for the code, analysis and drafting. The author directed the study, reviewed the results and edited the final text.

References

- [1] yosh95. *g4-developer-agent-paper*: per-task Gemma 4 31B outcomes on public competition tasks. GitHub, 2026. Apache-2.0.
- [2] C. Ge, D. Kryvosheieva, D. Fried, U. Girit, K. Hariharan. Agent Psychometrics: Task-Level Performance Prediction in Agentic Coding Benchmarks. arXiv:2604.00594, 2026.
- [3] N. Balepur, K. Tomlinson, T. Schnabel. Code Understanding is a Bottleneck for Coding Agents. arXiv:2610.10610, 2026.
- [4] C. E. Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR 2024; OpenAI, SWE-bench Verified, 2024.
- [5] J. Yang et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. NeurIPS 2024; mini-SWE-agent, 2025.
- [6] Kaggle competition forum, Gemma 4 Developer Agent Paper Track: "27 fastapi tasks cannot pass in the provided sandbox", Nov 2026.
- [7] C. S. Xia, Y. Deng, S. Dunn, L. Zhang. Agentless: Demystifying LLM-based Software Engineering Agents. arXiv:2407.01489, 2024.
- [8] Z. Chen et al. LocAgent: Graph-Guided LLM Agents for Code Localization. ACL 2025.
- [9] veljjanoski. *Better calls, not more calls: what limits a small coding agent on a fixed budget.* Gemma 4 Developer Agent Paper Track, Nov 2026; github.com/veljjanoski/gemma4-developer-agent.